ShelfFlow: Local-First Agentic AI Decision Support System

New MVP Proposal for the Final-Year Project

Proposed project type: Agentic AI decision-support upgrade for existing supermarket stock workflows
Primary users: Supermarket store managers and inventory staff
Pilot scope: One organization, one or two stores, and one perishable product category
Implementation strategy: Build and validate locally first; migrate selected components to AWS only when credits or university support are available
Core idea: Import existing stock data, identify batch expiry risk, explain recommended actions and show risk on a simple store layout.
Human-control rule: The agent may analyze, calculate, prioritize and propose. A manager must approve operational actions.

1. Project overview

This project is a **local-first AI upgrade to an existing supermarket workflow**, rather than a complete replacement for a supermarket-management platform. The application will be designed with clear service interfaces so that selected components can later migrate to AWS without rewriting the business logic.

A supermarket already has some way to record stock, batches, expiry dates and store locations. The FYP does not need to replace that system. Instead, the MVP accepts a structured CSV export and adds an Agentic AI layer that helps a manager interpret the data and decide what to do.

The new system will not attempt to build a complete POS, ERP, warehouse-management or procurement platform. It will demonstrate how an agent can use existing data and controlled tools to:

1. Find products and batches at risk.
2. Calculate and explain why they are at risk.
3. Compare possible actions such as transfer, discount review or reorder.
4. Notify a manager about important issues.
5. Simulate a product-placement change on a 2D store layout.
6. Ask for approval before an operational action is recorded.

This makes the contribution more specific than “a website for managing inventory.” The project becomes:

> **An Agentic AI decision-support upgrade that converts existing supermarket stock data into explainable expiry-risk and store-layout recommendations.**

2. Proposed research problem

Supermarket data may already exist in spreadsheets or operational systems, but managers still need to interpret several pieces of information together:

- Current quantity
- Batch and expiry date
- Historical sales
- Store demand
- Supplier or transfer lead time
- Shelf location
- Previous waste and stock-out records

A normal dashboard can display these values, but it may not explain which issue deserves attention first or what action should be considered.

The MVP investigates whether an Agentic AI interface can help a manager complete stock-risk tasks more effectively than a basic data table or rule-only dashboard.

Proposed research question

> **How can an AWS-based Agentic AI Decision Support System use existing supermarket stock data to identify expiry risk, generate explainable action recommendations and support store-layout planning while keeping the manager in control of operational decisions?**

Supporting questions

1. Can the agent correctly identify high-risk batches from imported data?
2. Can the agent explain its recommendation using the underlying data and calculation?
3. Can the agent choose the correct analysis tool for a manager’s question?
4. Does the agent reduce the time needed to find and understand stock risks compared with a basic inventory table?

5. Can the system prevent the agent from executing an unapproved transfer, disposal or layout change?
6. Can the heatmap help managers understand where risk is located inside the store?

3. What makes the system agentic?

The system should not be called agentic only because it contains a chatbot. The agent must be able to perform a controlled loop:

Plain Text

```
Understand manager request
        ↓
Select the required analysis tool
        ↓
Retrieve stock, batch, expiry and sales data
        ↓
Calculate risk or compare actions
        ↓
Explain the evidence and propose an action
        ↓
Ask for approval when an operational action is involved
        ↓
Record the decision and result
```

For example, a manager could ask:

> “Which yogurt batches in Store 1 may expire within seven days, and what should I review first?”

The agent should:

1. Identify the selected store and date window.
2. Call the expiry-risk analysis tool.
3. Retrieve the relevant batch and demand data.
4. Rank batches by estimated surplus at expiry.
5. Return the batch IDs, calculations and explanation.
6. Offer a possible transfer or discount review.
7. Ask for manager approval before recording an action.

The agent should not directly change stock quantities, create a purchase order, dispose of a product or permanently change a layout without an approval step.

4. MVP boundaries

Included

- Authentication and organization/store access
- CSV import with validation
- Batch-level stock and expiry records
- Historical sales data import
- Expiry-risk calculation
- A demand estimate sufficient for short-horizon risk analysis
- Dashboard with store and batch risk
- Agentic natural-language decision support
- Notifications and recommendations
- 2D Store Heatmap Planner
- Proposed layout simulation
- Manager approval and audit log
- AWS deployment architecture
- Evaluation against a non-agent baseline

5. The six MVP modules

The six MVP modules form one focused workflow. The Agentic AI capability is implemented mainly through the **Notification & Recommendation** module, supported by the other modules as tools and data sources.

Module 1: Authentication and Organization Management

Purpose

This module controls who can use the system and which organization/store data they can access. It is not only a login screen. It provides the security boundary required for an agent that can read operational data and create recommendations.

Main functions

- Authenticate a user using email.

- Send a first-time user to onboarding after successful authentication.
- Collect the user profile and preferred display name.
- Allow the user to create a new organization.
- Allow the user to join an existing organization using a join code.
- Assign a role such as owner, manager or staff.
- Add one or more stores to an organization.
- Select the active store when using the dashboard.
- Restrict every data query by organization and store scope.
- Record important user actions in an audit log.
- Identify which human approved or rejected an agent recommendation.

Inputs

- Email address
- User name
- Organization name when creating a new organization
- Invitation or join code when joining
- Store name
- User role
- Active store selection
- Approval or rejection identity

Processing

1. The authentication service verifies the user.
2. The system checks whether the user has completed onboarding.
3. A new user is redirected to `/onboarding` instead of directly to the dashboard.
4. The user chooses **Create organization** or **Join organization**.
5. When creating an organization, the system creates the organization, makes the user the owner and generates a unique join code.
6. When joining an organization, the system normalizes the submitted code, compares its secure hash with active organization codes and creates a membership after a successful match.
7. The API applies organization and store filters to downstream queries.

8. The authorization layer checks whether the requested operation is allowed.
9. The audit service records sensitive actions.

Outputs

- Authenticated user session
- Organization ID
- Active store ID
- User role
- Authorized dashboard data
- Permission-denied response when access is invalid
- Audit record for approvals, imports, edits and agent actions

Agent interaction

The agent receives a scoped user identity and store context. It must never be allowed to choose another organization by inventing an ID. The tool layer should verify the organization and store scope again before executing a tool.

Onboarding and join-code design

The login flow is:

```
Plain Text
/login → authenticate → /onboarding → create or join organization → /dashboard
```

Returning users who already completed onboarding go directly to the dashboard after authentication.

For the MVP, generate an organization join code using a cryptographically secure random generator. Use an unambiguous alphabet such as `ABCDEFGHJKLMNPQRSTUVWXYZ23456789`, which removes characters that are easy to confuse. Generate eight characters, normalize user input to uppercase, and enforce a unique database constraint. If a collision occurs, generate another code and retry. The code should be revocable and regeneratable by the organization owner.

Store a hash of the join code rather than relying only on plaintext storage. The organization owner can view or regenerate the current code. A future version can add one-time invitation codes with expiry dates.

Use email authentication and simple roles. Do not build a complex enterprise identity system. Amazon Cognito can provide the AWS identity layer, while the application database stores onboarding status, organization membership and roles.

Module 2: Data Import from CSV

Purpose

This module upgrades the retailer’ s existing workflow instead of requiring a new POS or ERP. The manager exports or prepares CSV files, uploads them, reviews validation errors and confirms the import.

The CSV module is important academically because it demonstrates integration with an existing data workflow. The FYP is not asking the store to replace its current system.

Supported CSV files

The MVP should support four file types:

1. Product master
2. Store and location master
3. Batch stock
4. Daily sales history

Suggested product CSV columns

Plain Text

```
sku, barcode, product_name, category, unit,
storage_type, shelf_life_days, supplier_code
```

Suggested store/location CSV columns

Plain Text

```
store_code, store_name, location_code,
location_name, location_type, storage_type,
capacity_units
```

Suggested batch CSV columns

Plain Text

```
store_code, sku, batch_code, received_date,
expiry_date, received_quantity, remaining_quantity,
reserved_quantity, status
```

Suggested sales CSV columns

Plain Text

```
store_code, sku, sales_date, units_sold,
available_stock, stockout_flag, promotion_flag
```

Main functions

- Upload a CSV file.
- Detect the file type or let the user select it.
- Map incoming columns to the required fields.
- Validate dates, quantities, identifiers and duplicates.
- Preview valid and invalid rows.
- Explain validation errors in plain language.
- Allow the user to correct or remove invalid rows.
- Import the confirmed rows.
- Record the import file, user and timestamp.
- Reject duplicate or conflicting imports safely.

Inputs

- CSV file
- File type selection
- Column mapping
- Organization and store scope
- Import confirmation
- Optional replacement or append mode

Validation rules

- Required columns must exist.

- Dates must use a recognized format.
- Expiry date must not be earlier than receiving date.
- Quantities must not be negative.
- Product and store identifiers must exist or be explicitly created.
- Duplicate batch codes must be flagged.
- Sales dates must not be in an invalid future period.
- Storage type must be compatible with the product and location configuration.

Outputs

- Import preview
- Valid-row count
- Invalid-row count
- Row-level error messages
- Import job ID
- Imported products, stores, batches or sales records
- Import audit record
- Data-quality warning for incomplete or suspicious data

Agent interaction

The agent can explain an import error or summarize the imported data, but the import engine itself should remain deterministic. For example:

> “The import contains 12 valid batch rows and 3 invalid rows. Two rows have an expiry date before the received date. One row refers to an unknown store code.”

The agent should not silently repair business data.

MVP limitation

Support one agreed CSV schema plus a simple column-mapping screen. Do not build a universal importer for every supermarket system.

Module 3: Batch System

Purpose

This module tracks stock as batches rather than treating all units of a product as identical. This is required because two batches of the same product may have different expiry dates and different risk.

Example:

```
Product: Milk 1L
Batch 11: 40 units, expires in 3 days
Batch 12: 100 units, expires in 12 days
```

A product-level quantity of 140 units would hide the important difference between these batches.

Main functions

- Create a batch when imported or received.
- Store batch code, product, store and supplier.
- Store received quantity and remaining quantity.
- Store received date and expiry date.
- Track reserved, available, damaged, donated and disposed quantities.
- Record stock movements without overwriting history.
- Calculate remaining shelf life.
- Link sales or adjustments to a batch when the data supports it.
- Show batches sorted by expiry risk.

Inputs

- Product ID or SKU
- Store ID
- Batch code
- Supplier ID, if available
- Received date
- Expiry date
- Received quantity
- Remaining quantity

- Reserved quantity
- Stock movement type
- Movement quantity
- User or import job ID

Processing

The batch service calculates:

```
Plain Text

remaining_days = expiry_date - current_date
available_quantity = remaining_quantity - reserved_quantity
```

When the user records a stock movement, the system creates an immutable transaction and updates the batch summary. The first MVP should use a ledger approach so the history can be audited.

Outputs

- Batch list
- Batch detail page
- Current quantity by status
- Remaining days
- Stock movement history
- Expiry-risk input data
- Traceable audit record

Agent interaction

The agent can query batches through a read-only tool:

```
Plain Text

get_batches(store_id, product_id, expiry_window)
```

It may then pass the results to the risk-analysis tool. It must not directly modify a batch through a free-form database query.

MVP limitation

Use FIFO-like batch consumption in the simulated dataset if exact batch-level sales allocation is unavailable. Clearly label this as an assumption. Do not claim that historical sales are perfectly attributable to individual batches unless the source data supports it.

Module 4: Dashboard

Purpose

The dashboard is the visual operating screen for managers. It summarizes the imported data and shows which issue deserves attention first. It should be understandable without asking the AI agent a question.

Main functions

- Show selected organization and store.
- Display total products and active batches.
- Show batches approaching expiry.
- Show high-risk batch count.
- Show stock-out and low-stock indicators.
- Show estimated surplus at expiry.
- Show pending recommendations and approvals.
- Filter by store, category, product, expiry window and risk level.
- Drill down from a summary card to the exact batch.
- Open the heatmap for a selected store.
- Display data-quality and stale-data warnings.

Inputs

- Organization scope
- Store selection
- Date range
- Category filter
- Risk filter
- Batch data
- Sales data
- Forecast or demand estimate

- Recommendation status
- Layout positions

Processing

The dashboard reads calculated results. It should not perform hidden calculations only in the browser. The backend risk service should return:

- Risk level
- Risk score or estimated surplus
- Forecast input window
- Data timestamp
- Explanation

Outputs

- Risk summary cards
- Expiry-risk table
- Stock and batch charts
- Pending-action list
- Store comparison for authorized users
- Links to batch details and heatmap
- Data-quality warnings

Agent interaction

The dashboard can include an agent panel where a manager asks questions such as:

- “Show batches likely to expire within seven days.”
- “Why is Batch 12 red?”
- “Which store may be able to receive this stock?”
- “Summarize today’s urgent risks.”

The agent response should link to the dashboard records used as evidence.

MVP limitation

Use a small number of clear indicators. Do not add dozens of charts. The dashboard’s main purpose is to help the manager identify and understand the next issue.

Module 5: Notification and Recommendation

Purpose

This module is the central Agentic AI upgrade. It does two related jobs:

1. Notify a manager about important risks.
2. Use an agent to investigate the risk and propose an explainable action.

The agent is not the source of truth. The database, deterministic calculations and controlled tools are the source of truth. The agent decides which authorized tool to call and how to explain the result.

Notification functions

- Create an alert when a batch enters a configured risk level.
- Create an alert when stock may be insufficient.
- Create an alert when an import finishes with errors.
- Create an alert when an approval is pending.
- Mark an alert as read.
- Acknowledge an alert.
- Escalate an unreviewed alert in the prototype.

Recommendation functions

- Rank urgent risks.
- Explain why a batch is at risk.
- Suggest transfer review.
- Suggest discount review.
- Suggest donation or disposal review where authorized.
- Suggest reorder review for low projected stock.
- Suggest no action when the data does not support an urgent action.
- Ask for missing information.
- Record manager approval or rejection.

Inputs

The agent may receive:

- Manager’ s natural-language request
- User and organization scope
- Store selection
- Current date
- Batch data
- Expiry dates
- Remaining quantity
- Sales history or demand forecast
- Supplier lead time
- Other-store stock and demand
- Store-layout data
- Historical waste and stock-out data
- Store policy documents, if included in the knowledge base

Agent tools

The agent should be allowed to call a small, explicit set of tools:

Plain Text

```
get_store_summary(store_id)
get_expiry_risks(store_id, expiry_window, risk_level)
get_batch_detail(batch_id)
calculate_demand(product_id, store_id, horizon_days)
calculate_reorder_review(product_id, store_id)
find_transfer_candidates(batch_id)
simulate_product_placement(layout_id, product_id, target_location_id)
create_notification(alert_type, target_user_id, evidence_ids)
create_recommendation(recommendation_type, evidence_ids, reason)
request_manager_approval(recommendation_id)
record_manager_decision(recommendation_id, decision, reason)
```

Each tool should validate its parameters and organization scope. The tools should return structured JSON rather than unstructured text.

Example agent workflow: expiry review

Manager request:

> “Find the urgent expiry problems in Store 1 and tell me what I should review first.”

Agent execution:

1. Calls `get_store_summary`.
2. Calls `get_expiry_risks` with a seven-day window.
3. Calls `get_batch_detail` for the highest-risk batch.
4. Calls `calculate_demand` for the affected product.
5. Calls `find_transfer_candidates` if a transfer review is relevant.
6. Creates a recommendation draft.
7. Displays evidence and calculation.
8. Requests manager approval if the manager wants to record the action.

Recommendation output

Every recommendation should contain:

JSON

```
{
  "recommendationId": "rec-1008",
  "type": "transfer_review",
  "storeId": "store-01",
  "batchId": "batch-12",
  "suggestedQuantity": 20,
  "priority": "high",
  "reason": "The batch may have 40 units remaining at expiry.",
  "evidence": [
    "batch-12",
    "forecast-2026-11-04",
    "inventory-store-01"
  ],
  "assumptions": [
    "Forecast uses the last 28 usable sales days.",
    "Transfer time is assumed to be one day."
  ],
  "requiresApproval": true,
  "status": "draft"
}
```

Agent autonomy levels

Use three levels so the project is academically clear and safe:

Level 1 — Observe: The agent reads data and summarizes risks.

Level 2 — Recommend: The agent calls analysis tools and creates a recommendation draft.

Level 3 — Execute: The system records or performs an operational action after explicit manager approval.

The MVP should implement Levels 1 and 2 fully, and implement Level 3 only as a simulated or approved record. It should not connect the agent directly to a real purchase, disposal or transfer system.

Outputs

- In-app notification
- Natural-language explanation
- Structured recommendation
- Evidence links
- Assumptions and data timestamp
- Approval request
- Approval or rejection record
- Agent tool trace for evaluation

AWS AgentCore implementation

Use Amazon Bedrock AgentCore as the current AWS agentic platform. AWS describes AgentCore as supporting agent deployment, tools, permissions, governance and observability, and it can work with different frameworks and foundation models. [1]

A practical MVP arrangement is:

- **AgentCore Runtime:** hosts the agent code and isolates sessions.
- **AgentCore Gateway:** exposes approved APIs or Lambda functions as agent tools.
- **Amazon Bedrock model:** interprets the manager request and produces the explanation.
- **AgentCore Observability or CloudWatch:** records tool calls, latency and failures.
- **AgentCore Memory:** optional; use only for short conversational context if needed.

AWS documentation explains that AgentCore Runtime is a managed environment for agents and tools, while Gateway can expose APIs and Lambda functions as MCP-compatible tools. [2] [3]

Do not build the MVP using the legacy Bedrock Agents Classic workflow without checking AWS account availability. AWS documentation states that Bedrock Agents Classic is in

maintenance mode and directs new customers toward AgentCore. [4]

Guardrails and approval

Use AWS IAM and tool-level authorization to control what the agent can call. Use Bedrock Guardrails where appropriate for unsafe input/output filtering and grounding checks. AWS describes Guardrails as configurable safeguards for model inputs and outputs, including sensitive-information filters and contextual-grounding checks. [5]

The most important safeguard for this FYP is not only a language-model guardrail. It is a business-rule guardrail:

```
Plain Text

Agent proposes action
        ↓
Manager approves
        ↓
System records approved action
```

MVP limitation

Use one agent with a small tool set. Do not build multiple cooperating agents. Do not let the model perform calculations that can be done reliably by code. The agent should orchestrate tools and explain their results.

Module 6: Store Heatmap Planner

Purpose

This module lets a manager create a simple store representation instead of requiring the system to know the store layout in advance. The manager first configures the number of grid rows and columns, then drags products into the appropriate shelf or storage position.

The recommended representation is a **two-level grid**:

```
Plain Text

Store layout
└── Grid cell / fixture position
    └── Shelf level or row
        └── One or more product placements
```

A grid cell represents a physical fixture position such as a shelf bay, refrigerator section or freezer section. Each cell can contain multiple shelf levels. Each shelf level can contain

multiple products represented as compact product chips. This supports the case where one shelf contains several rows and several products without requiring a complicated floor-plan editor.

Why this method is suitable for the MVP

A completely free-form floor-plan editor would increase the scope significantly. A configured grid is easier to implement, easier for a manager to understand and easier to evaluate. It still supports meaningful layout planning because products have a store position, shelf level, capacity and operational indicators.

The system should not claim that the grid automatically represents exact physical measurements. It is a planning abstraction. The manager is responsible for configuring the grid to match the store.

Manager configuration flow

1. The manager creates a layout for a selected store.
2. The manager enters the number of grid rows and columns.
3. The system generates empty cells with coordinates such as `R01-C01`.
4. The manager names or labels zones, aisles or fixtures if needed.
5. The manager selects the cell type: shelf, refrigerator, freezer, display or other.
6. The manager sets the number of shelf levels for each cell.
7. The manager optionally sets capacity and storage requirements.
8. The manager drags products into shelf levels.
9. The manager can place multiple product chips on one shelf level.
10. The manager saves the layout as a draft and activates it after review.

Main functions

- Create a layout for a store.
- Configure grid rows and columns.
- Create, rename and remove grid cells.
- Configure the number of shelf levels inside a cell.
- Label each cell and shelf level.
- Select a cell type and storage type.
- Drag products between shelf levels.

- Place multiple products on one shelf level.
- Display demand, expiry, waste and stock-out indicators.
- Check storage compatibility and configured capacity.
- Save draft layout versions.
- Compare the active layout with a proposed layout.
- Activate a layout only after manager confirmation.

Inputs

- Store ID
- Layout name
- Layout version
- Number of grid rows
- Number of grid columns
- Cell coordinate
- Cell label and type
- Number of shelf levels per cell
- Shelf-level label
- Product ID
- Product position within a shelf level
- Capacity
- Storage type
- Product storage requirement
- Demand estimate
- Expiry-risk result
- Waste history
- Stock-out history
- Proposed target cell and shelf level

Suggested data structure

JSON

```
{
  "layoutId": "layout-store-01-v3",
  "storeId": "store-01",
  "grid": { "rows": 4, "columns": 6 },
  "cells": [
    {
      "cellId": "R01-C01",
      "label": "Chilled A1",
      "cellType": "refrigerated_shelf",
      "shelfLevels": [
        {
          "levelId": "R01-C01-L01",
          "label": "Top row",
          "capacityUnits": 80,
          "products": [
            { "productId": "yogurt-strawberry-01", "displayOrder": 1 },
            { "productId": "yogurt-blueberry-01", "displayOrder": 2 }
          ]
        }
      ]
    }
  ]
}
```

Visual design recommendation

Render each grid cell as a fixture card. Divide the card into horizontal shelf-level rows. Render each product as a compact chip inside the shelf-level row.

Plain Text

```
┌──────────────── Cell R01-C01 ────────────────┐
| Top row:   [Yogurt A] [Yogurt B]             |
| Middle row: [Milk A]                         |
| Bottom row: [Empty]                          |
└──────────────────────────────────────────────┘
```

The grid cell or product chip can use indicators without relying on one confusing color:

- **Demand border:** relative demand compared with other products in the category.
- **Expiry fill or warning icon:** batch expiry-risk level.
- **Waste icon:** recorded waste above a configured threshold.
- **Stock-out icon:** recorded stock-out event or day.
- **Warning marker:** storage or capacity incompatibility.

- **Insufficient-data marker:** shown when there is not enough historical data.

The manager should be able to select an indicator legend. The system should not imply that a color alone is a recommendation.

Proposed-layout simulation

When a manager drags a product to another cell or shelf level, the system should:

1. Copy the active layout into a draft version.
2. Move the product to the proposed cell and shelf level.
3. Check storage compatibility.
4. Check the configured capacity.
5. Recalculate the visual indicators for the proposed position.
6. Compare current and proposed positions.
7. Return warnings and evidence.
8. Ask for manager confirmation before activating the layout.

Inputs to the simulation

JSON

```
{
  "layoutId": "layout-store-01-v3",
  "productId": "yogurt-strawberry-01",
  "currentCellId": "R01-C01",
  "currentLevelId": "R01-C01-L01",
  "targetCellId": "R02-C03",
  "targetLevelId": "R02-C03-L02",
  "simulationDate": "2026-11-01"
}
```

Simulation output

JSON

```
{
  "simulationId": "sim-2201",
  "valid": true,
  "warnings": [
    "The target shelf level has 15% less configured capacity."
  ],
  "currentPosition": "R01-C01-L01",
```

```
  "proposedPosition": "R02-C03-L02",
  "currentIndicators": {
    "demand": "high",
    "expiryRisk": "high",
    "waste": "medium"
  },
  "proposedIndicators": {
    "demand": "high",
    "expiryRisk": "medium",
    "waste": "medium"
  },
  "explanation": "The proposed position has higher recorded demand and may redu
  "requiresApproval": true
}
```

Agent interaction

The agent may call the heatmap simulation tool when a manager asks:

> “What happens if I move the high-risk yogurt batch to the front row of the chilled shelf?”

The agent should summarize the simulation result and warnings. It must not claim causation or activate the layout without manager confirmation.

MVP limitation

Use one configured 2D grid per store. Do not build 3D modeling, computer vision, automatic shelf detection or exact physical measurement conversion.

6. AWS system architecture

Cost reality and zero-cost-first strategy

The project should not promise that Amazon Bedrock or AgentCore will be free. Amazon Bedrock charges according to the model and token usage, and Amazon Bedrock AgentCore uses consumption-based pricing. AWS’ s current Free Tier page also lists AgentCore as a paid-plan-exclusive service. [8] [9] [10]

Therefore, the MVP will use two compatible implementation modes:

Mode A — zero-cost development and evaluation: Run the web application, PostgreSQL database and agent adapter locally. Use OpenCode as the primary agent provider for development and evaluation, with GLM-5.3-Flash Free when it is available under OpenCode’ s current free offering. OpenCode documents this model as free for a limited time, so the proposal treats its availability as a variable rather than a permanent guarantee. [11] OpenCode also supports multiple providers and local models, which keeps the provider

adapter replaceable. [12] Keep a deterministic mock agent as a fallback for tests and for periods when the free model is unavailable. This mode does not require Bedrock or AgentCore API calls. It is the required fallback so the project can be completed without an AWS bill.

Mode B — AWS agent demonstration: When AWS student credits, university credits or an approved departmental account are available, connect the same agent interface to Amazon Bedrock and AgentCore Runtime/Gateway for a limited demonstration. Set a budget alert, restrict the dataset and limit the number of agent calls.

The application should define a provider-independent interface:

```
Plain Text

AgentProvider
├── OpenCodeAgentProvider
├── MockAgentProvider
└── BedrockAgentCoreProvider
```

The tool contracts, database, approval rules and evaluation dataset remain the same in both modes. This means the project is still architecturally AWS-ready without making paid AWS AI services a dependency for basic completion.

Free-first deployment recommendation

For the zero-cost MVP, develop with:

- Next.js, TypeScript and Tailwind CSS locally
- Bun and Elysia locally
- PostgreSQL in Docker locally
- Local object storage or local filesystem for CSV development files
- OpenCode agent provider using GLM-5.3-Flash Free when available
- Deterministic mock agent for repeatable tests and fallback

If a cloud demonstration is required without using paid AI services, use AWS services only within their current Free Tier limits and verify the account’s quotas before deployment. AWS states that its Free Tier includes selected services within limits, but the plan, service and account conditions matter. [10]

The proposal should never state “AWS deployment is free” without identifying the exact account plan, service, region and quota. Bedrock and AgentCore usage should be treated as optional funded demonstrations.

Local-to-AWS migration path

The project will be implemented in this order:

1. Build the complete business workflow locally.
2. Test the deterministic tools with seeded data.
3. Test the OpenCode agent adapter, mock fallback and approval rules.
4. Containerize or package the web, API and agent interfaces.
5. Deploy only the components supported by the available AWS account.
6. Replace the OpenCode agent adapter with Bedrock/AgentCore only if credits or institutional funding are available.
7. Compare local and AWS-mode outputs using the same evaluation dataset.

The migration target is an implementation option, not a prerequisite for demonstrating the research contribution.

Recommended services

Amazon Cognito handles user authentication. The application maps users to organizations, roles and stores.

AWS Amplify Hosting or an equivalent AWS web-hosting route can host the Next.js interface. If the existing preferred Next.js/TypeScript/Tailwind stack is retained, the frontend remains unchanged.

Amazon API Gateway exposes the application API.

AWS Lambda runs small backend functions for CSV validation, batch queries, risk calculations, dashboard summaries, recommendation records and tool functions. AWS describes Lambda as supporting direct event invocation from services such as API Gateway, S3 and EventBridge. 6

Amazon RDS for PostgreSQL can store relational organization, batch, inventory, sales, recommendation, layout and audit data in the funded AWS mode. For the zero-cost mode, use local PostgreSQL. Do not create an RDS instance until the account’ s free allowance or credits have been confirmed.

Amazon S3 stores uploaded CSV files, import results, generated exports and optional layout assets.

Amazon Bedrock AgentCore Runtime can host the agent in the funded AWS mode. It is not required for local MVP completion.

Amazon Bedrock AgentCore Gateway can expose the approved backend APIs and Lambda functions as tools in the funded AWS mode. In the zero-cost mode, the OpenCode

agent adapter calls the same tool functions directly.

Amazon Bedrock can supply the foundation model used to interpret requests and generate explanations in the funded AWS mode. In the zero-cost mode, use the OpenCode agent adapter or a deterministic mock for development and evaluation.

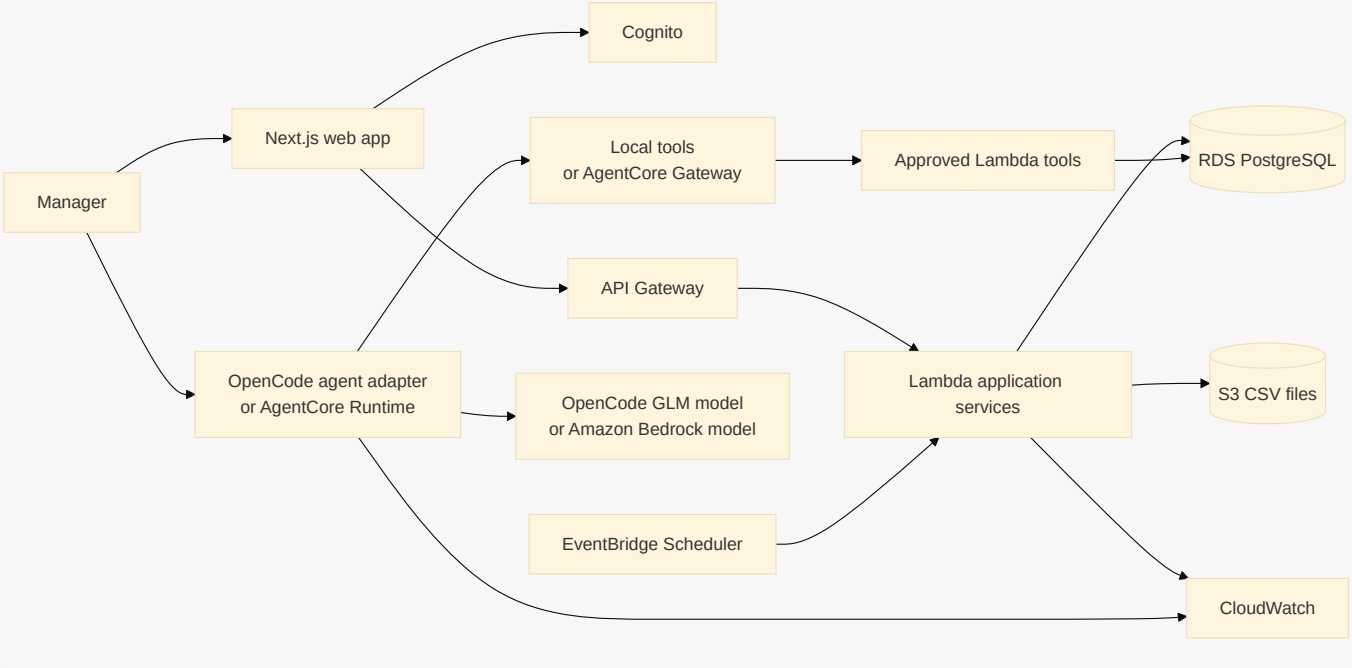
Amazon EventBridge Scheduler can trigger a daily expiry-risk scan.

AWS Step Functions is optional. Use it only if the scheduled scan becomes a multi-step workflow that needs visible retries and state. For the first implementation, a single scheduled Lambda or a small Step Functions workflow is enough.

Amazon CloudWatch stores logs, metrics and alerts.

Architecture flow

mermaid



Why the agent should not query the database directly

The agent should call approved tools rather than receive unrestricted database credentials. A tool can:

- Validate organization and store scope.
- Validate input parameters.
- Apply deterministic business rules.
- Return only the required fields.
- Log the call.

- Require approval for state-changing operations.

This also makes the system easier to evaluate because every agent action is visible.

7. Data model for the MVP

Use the following core entities:

```
Plain Text

users
organizations
organization_members
stores
store_locations
products
suppliers
batches
inventory_transactions
sales_records
risk_results
notifications
recommendations
layout_versions
product_positions
approval_records
import_jobs
import_errors
audit_logs
```

The key relationship is:

```
Plain Text

organization → stores → locations
organization → products → batches
store + product + date → sales records
batch + forecast → risk result
risk result → recommendation → approval record
store + layout version → product positions
```

8. API and tool design

Application API

Plain Text

```
POST /imports
GET  /imports/:id
POST /batches/query
GET  /dashboard/summary
GET  /risk/expiry
GET  /notifications
GET  /recommendations
POST /recommendations/:id/approve
POST /recommendations/:id/reject
GET  /stores/:id/layouts
POST /layouts/:id/simulate
POST /layouts/:id/approve
GET  /audit-logs
```

Agent tool boundary

Expose only the following tool categories to the agent:

- Read store and batch data
- Calculate risk
- Calculate demand estimate
- Find transfer candidates
- Simulate layout
- Draft notification
- Draft recommendation
- Request manager approval
- Record approved decision

Do not expose a generic SQL tool, unrestricted shell tool or generic “update anything” endpoint.

9. Academic evaluation plan

The main comparison should be between:

Baseline workflow: User searches a stock/batch table and manually interprets the expiry data.

Agentic workflow: User asks the agent, receives evidence-backed analysis, reviews the dashboard and approves or rejects the recommendation.

Test scenario

Use a controlled dataset containing:

- 30–50 products
- One or two stores
- Multiple batches per perishable product
- Different expiry dates
- Seven to thirty days of daily sales
- Some stock-outs
- Some surplus batches
- Some possible transfer candidates
- A simple store layout

Tasks

1. Find batches likely to expire within seven days.
2. Explain why the highest-risk batch is risky.
3. Identify a possible action.
4. Find whether another store could receive the batch.
5. Simulate a placement change.
6. Approve or reject the recommendation.

Measures

- Task completion time
- Correctly identified high-risk batches
- Correctly selected action for seeded scenarios
- Explanation-grounding score
- Number of unsupported claims
- Number of unnecessary tool calls
- Tool-call success rate
- Approval-control violations
- User confidence and perceived usefulness
- Agent response latency

- Estimated AWS cost per scenario

Agent-specific evaluation

Create a test set of manager questions and expected tool behavior. For each question, check:

- Did the agent identify the correct store?
- Did it call the correct tool?
- Did it pass valid parameters?
- Did it use the returned evidence?
- Did it mention assumptions?
- Did it avoid inventing stock or expiry values?
- Did it request approval before a state-changing operation?

Suggested success targets

These are FYP evaluation targets, not industry benchmarks:

- At least 90% of deterministic risk calculations match the seeded expected results.
- At least 80% of test questions call the correct tool on the first attempt.
- At least 80% of participants complete the main risk-review task without technical assistance.
- Zero unauthorized state-changing actions occur in the test scenarios.
- Participants can identify the evidence supporting a recommendation.
- The agent performs no unsupported action when data is missing.

10. Implementation roadmap for one year

Months 1–2: Scope, research and data design

- Confirm the research question with the supervisor.
- Confirm the six modules and MVP boundaries.
- Define CSV formats.
- Create the data dictionary.
- Prepare the controlled evaluation dataset.
- Define the agent tool contract.

- Review AWS account, region, service access and budget limits.

Months 3–4: Local existing-workflow upgrade foundation

- Build local email authentication and organization/store scope.
- Build CSV upload and validation.
- Build product, store, location and batch records.
- Run the basic web interface and API locally.

Months 5–6: Risk and dashboard

- Build inventory and sales queries.
- Implement demand estimate.
- Implement expiry-risk calculation.
- Build dashboard and batch drill-down.
- Build audit logging.

Months 7–8: Local agentic decision support

- Implement the provider-independent agent interface.
- Implement the OpenCode agent adapter using GLM-5.3-Flash Free when available, plus a deterministic mock agent.
- Implement approved read-only tools.
- Implement recommendation drafting.
- Add evidence and assumption output.
- Add notifications.
- Add approval and rejection workflow.
- Record local tool traces and evaluation logs.

Months 9–10: Store Heatmap Planner

- Build simple 2D layout.
- Add product positions.
- Add demand, expiry and waste indicators.
- Build layout simulation.
- Add capacity and storage warnings.

- Add approval and layout versioning.

Month 11: Evaluation and optional AWS migration

- Run deterministic test cases.
- Evaluate tool selection and grounding.
- Conduct user tasks with baseline and agentic workflow.
- Measure time, correctness, usability and approval safety.
- If credits are available, deploy a limited AWS-mode demonstration.
- Compare local-mode and AWS-mode outputs using the same test questions.

Month 12: Dissertation and final delivery

- Analyze results.
- Document limitations.
- Improve the final demonstration.
- Prepare architecture, evaluation and ethics sections.
- Present AWS migration and live Bedrock/AgentCore deployment as completed work only if it was actually funded and tested; otherwise document it as future work.

11. Main risks and controls

Risk: The agent produces an incorrect recommendation

Control: Use deterministic analysis tools for quantities and risk. The model explains results but does not invent them.

Risk: The agent performs an unauthorized operation

Control: Separate read-only tools from state-changing tools. Require manager approval and verify authorization in the backend.

Risk: The project becomes a complete ERP

Control: Use CSV as the integration boundary. Keep live POS, procurement and warehouse features out of the MVP.

Risk: Insufficient real data

Control: Use synthetic or anonymized data for development and clearly distinguish demonstration results from real operational validation.

Risk: AWS costs become too high

Control: Use a single development region, small datasets, scheduled rather than continuous jobs, CloudWatch budgets, limited agent calls and local mocks during development. AWS services used for agents, models, databases and hosting are not automatically zero-cost.

Risk: The agent is only a chatbot

Control: Evaluate tool selection, structured action calls, evidence use, approval behavior and measurable task outcomes.

Risk: Heatmap becomes only a visual gimmick

Control: Connect the heatmap to batch risk, demand, waste and stock-out evidence. Test whether users use it to make a different planning decision.

12. Recommended final project title

> **ShelfFlow: A Local-First, AWS-Compatible Agentic AI Decision Support System for Supermarket Inventory Risk and Store-Layout Planning**

Alternative shorter title:

> **An Agentic AI Decision Support System for Supermarket Expiry-Risk Management**

The shorter title is safer if the Store Heatmap Planner becomes a supporting feature rather than the main research contribution.

13. Recommended final MVP statement

> ShelfFlow is an AWS-compatible Agentic AI Decision Support System that upgrades an existing supermarket stock workflow. Managers import product, batch, sales and store-layout data from CSV files. The system calculates expiry risk and demand indicators, while an agent uses controlled tools to investigate risks, generate evidence-backed recommendations and notify managers. The zero-cost MVP can run locally with an OpenCode agent adapter and a provider-independent interface; an optional funded demonstration can connect the same tools to Amazon Bedrock and AgentCore. Managers can simulate product placement on a 2D store heatmap and approve or reject proposed actions. The system does not automatically execute irreversible operational decisions.

Appendix A: Recommended agent and system tools

The following tools are candidates for the MVP. The tool names describe controlled operations, not unrestricted database access. Every tool must enforce organization and store scope on the backend.

Read-only analysis tools

Tool	Purpose	Main input	Main output
`get-store-summary`	Summarize products, batches, stock and urgent issues for one store.	`storeId`	Counts, warnings, data timestamp.
`get-expiry-risks`	Find batches likely to have unsold stock near expiry.	`storeId`, `expiryWindowDays`, `riskLevel`	Ranked batch risks with evidence.
`get-batch-detail`	Retrieve one batch and its movement history.	`batchId`	Product, quantity, dates and transactions.
`get-product-sales-history`	Retrieve usable sales history for a product and store.	`productId`, `storeId`, date range	Daily sales and data-quality notes.
`calculate-demand-estimate`	Calculate a short-horizon demand estimate using the selected baseline.	`productId`, `storeId`, `horizonDays`	Estimate, method, assumptions and confidence notes.
`find-transfer-candidates`	Find stores that may be able to receive surplus stock.	`batchId`	Candidate stores, demand and capacity evidence.
`get-layout-position`	Find where a product is placed in the configured grid.	`storeId`, `productId`	Cell, shelf level and indicators.

Drafting and simulation tools

Tool	Purpose	Main input	Main output

`create-notification-draft`	Prepare a notification for manager review.	Alert type, evidence IDs, priority	Draft notification.
`create-recommendation-draft`	Create a structured recommendation without executing it.	Recommendation type, evidence IDs, reason	Draft recommendation with assumptions.
`simulate-product-placement`	Compare the current and proposed grid positions.	Layout ID, product ID, current position, target position	Validity, warnings and indicator comparison.
`simulate-reorder-review`	Estimate whether a product may need replenishment review.	Product, store, horizon and lead-time assumption	Suggested review quantity and evidence.

Approval and audit tools

Tool	Purpose	Main input	Main output
`request-manager-approval`	Present a draft action for explicit approval.	Recommendation ID	Pending approval record.
`record-manager-decision`	Record approval or rejection after checking user permission.	Recommendation ID, decision, reason	Final decision and audit record.
`activate-layout-version`	Make a reviewed layout draft active.	Layout version ID, manager confirmation	Active layout version and audit record.

Tool permissions

- Read-only analysis tools may be called during normal investigation.
- Drafting tools may create drafts but must not change operational stock.
- Approval tools require an authenticated manager or authorized role.
- `activate-layout-version` requires explicit confirmation.
- No tool may execute disposal, purchasing, stock transfer or financial actions in the MVP.
- Every tool returns structured JSON containing the data timestamp, organization scope and evidence IDs.

Appendix B: Page and module mapping

The following page map makes the location of each module explicit.

Page	Route	Module	Main responsibility
Login	`/login`	Authentication and Organization Management	Authenticate the user and route first-time users to onboarding.
Onboarding	`/onboarding`	Authentication and Organization Management	Complete profile, create an organization or join one with a code.
Dashboard	`/dashboard`	Dashboard; Notification and Recommendation	Show store summary, urgent risks, notifications, pending approvals and a link to the agent panel.
Store selector	`/stores`	Authentication and Organization Management	Select an authorized store and manage the active store context.
CSV import	`/imports`	Data Import from CSV	Upload files, map columns, validate rows, preview errors and confirm imports.
Import detail	`/imports/:importId`	Data Import from CSV	Show import status, row errors, warnings and audit information.
Batch list	`/inventory/batches`	Batch System	Filter batches by product, store, expiry window, quantity and risk.
Batch detail	`/inventory/batches/:batchId`	Batch System	Show batch dates, quantities, movements, risk evidence and related recommendations.
Recommendations	`/recommendations`	Notification and Recommendation	List, filter and prioritize agent-generated recommendation drafts.

Recommendation detail	`/recommendations/:recommendationId`	Notification and Recommendation	Show evidence, assumptions, tool trace summary and approve/reject controls.
Agent assistant	`/assistant`	Notification and Recommendation	Accept manager questions and display evidence-backed agent responses.
Store heatmap	`/stores/:storeId/layout`	Store Heatmap Planner	Configure grid dimensions, configure shelf levels and drag products into positions.
Layout simulation	`/stores/:storeId/layout/simulations/:simulationId`	Store Heatmap Planner	Compare current and proposed product positions and show warnings.
Product catalog	`/products`	Batch System; Data Import from CSV	Browse product master data and open related batches.
Notifications	`/notifications`	Notification and Recommendation	Review alerts, acknowledge them and open related risks.
Organization settings	`/settings/organization`	Authentication and Organization Management	Manage organization name, stores, roles and join-code regeneration.
Audit log	`/settings/audit-log`	Authentication and Organization Management; Notification and Recommendation	Review imports, approvals, rejected actions and layout activations.

Suggested page navigation

Plain Text

```
/login
  ├── first-time user → /onboarding
```

```
  └── onboarded user → /dashboard

/onboarding
  ├── create organization → /stores → /dashboard
  └── join organization → /stores → /dashboard

/dashboard
  ├── urgent risk → /inventory/batches/:batchId
  ├── recommendation → /recommendations/:recommendationId
  ├── import data → /imports
  └── plan layout → /stores/:storeId/layout
```

Appendix C: Onboarding and organization join-code flow

Create organization

1. The user completes authentication.
2. The user enters a display name and selects **Create organization**.
3. The user submits an organization name and first store name.
4. The backend creates the organization and store in one transaction.
5. The backend creates the user’s organization membership with the `owner` role.
6. The backend generates the join code using a secure random generator.
7. The frontend displays the organization name, code and next step.
8. The user is redirected to the dashboard.

Join organization

1. The user completes authentication.
2. The user completes the profile and selects **Join organization**.
3. The user enters the join code.
4. The backend normalizes the code to uppercase and removes accidental spaces.
5. The backend compares the submitted code hash with active organization codes.
6. The backend checks that the user is not already a member.
7. The backend creates a membership with the default `staff` role.
8. The organization owner can later change the role.
9. The user selects an authorized store and is redirected to the dashboard.

Join-code pseudocode

Plain Text

```
alphabet = ABCDEFGHJKLMNPQRSTUVWXYZ23456789

createJoinCode():
    repeat:
        code = secureRandomCharacters(alphabet, 8)
        normalized = uppercase(code)
        hash = hashJoinCode(normalized)
    until no active organization has hash
    store hash with organization and created timestamp
    return normalized
```

The code is not a replacement for authorization. Every request still checks the authenticated user’ s organization membership and role.

References

[1] Amazon Bedrock AgentCore Overview
[2] Host Agents or Tools with Amazon Bedrock AgentCore Runtime
[3] Amazon Bedrock AgentCore Gateway
[4] How Amazon Bedrock Agents Works and AgentCore Migration Note
[5] Amazon Bedrock Guardrails
[6] Creating Event-Driven Architectures with AWS Lambda
[7] Amazon Bedrock Knowledge Bases
[8] Amazon Bedrock Pricing
[9] Amazon Bedrock AgentCore Pricing
[10] AWS Free Tier
[11] OpenCode Models and Free Models
[12] OpenCode Providers and Local Model Support
[13] OpenCode Agents and Permissions